Assessment Series. -Day 1(A)

Contest Problem Report

Contest Code	GLBAVF
Total Problems	5
Coding Problems	5

Problem Name: Convert the String

Problem Statement:

Vivek was quite bored with the lockdown, so he came up with an interesting task. He successfully completed this task and now, he would like you to solve it.

You are given two strings A and B, each with length N. Let's index the characters in each string from — for each i ($0 \le i \lt N$),the $i+1$-th characters of A and B are denoted by A_i and B_i respectively. You should convert A to B by performing operations of the following type:

- Choose a subset S of the set $\{0, 1, ..., N-1\}$.
- Let c be the alphabetically smallest character among A_x for all $x \in S$.
- For each $x \in S$, replace A_x by c.

You should find the smallest necessary number of operations or report that it is impossible to convert A to B. If it is possible, you also need to find one way to convert A to B using this smallest number of operations. If there are multiple solutions, you may find any one.

Input

- The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows.
- The first line of each test case contains a single integer N.
- The second line contains a single string A.
- The third line contains a single string B.

Output

For each test case:

- If it is impossible to convert A to B, print a single line containing the integer -1.
- Otherwise, first, print a line containing a single integer K — the minimum number of operations.
- Then, print K lines describing the operations. Each of these lines should contain a positive integer Z, followed by a space and Z pairwise distinct space-separated integers from the set $\{0, 1, ..., N-1\}$ — the elements of S.

Constraints

- $1 \le T \le 20$
- $1 \le N \le 10^3$
- $|A| = |B| = N$
- A and B contain only lowercase English letters

Subtasks

Subtask #1 (30 points): B contains only characters 'a' and 'b'

Subtask #2 (70 points): original constraints

Sample Input 1:

3 5 abcab aabab 3 aaa aab 2 de cd

Sample Output 1:

2 3 1 2 4 3 0 1 3 -1 -1

Explanation:

Example case 1:

- First, we can choose $S = (1, 2, 4)$, so the character c is 'b' and the string A after this operation is "abbab".
- Then, we choose $S = (0, 1, 3)$, so c is 'a', and A becomes "aabab".
- There is no way to convert A to B in only one operation.

Example case 2: We can see that it is impossible to convert A to B since c is always 'a'.

Example case 3: We can see again that it is impossible to convert A to B since c cannot be 'c'.

Problem Name: Chef and Hamming Distance of arrays

Problem Statement:

Read problems statements in Vietnamese

.

Chef likes to work with arrays a lot. Today he has an array **A** of length **N** consisting of positive integers. Chef's little brother likes to follow his elder brother, so he thought of creating an array **B** of length **N**. The little brother is too small to think of new numbers himself, so he decided to use all the elements of array **A** to create the array **B**. In other words, array **B** is obtained by shuffling the elements of array **A**.

The little brother doesn't want Chef to know that he has copied the elements of his array **A**. Therefore, he wants to create the array **B** in such a way that the *Hamming distance* between the two arrays **A** and **B** is maximized. The Hamming distance between **A** and **B** is the number of indices **i** ($1 \leq \mathbf{i} \leq \mathbf{N}$) such that $\mathbf{A_i} \neq \mathbf{B_i}$.

The brother needs your help in finding any such array **B**. Can you please find one such array for him?

Note that it's guaranteed that no element in **A** appears more than twice, i.e. frequency of each element is at most 2.

Input

- The first line of the input contains a single integer **T** denoting the number of test cases. The description of **T** test cases follows.
- The first line of each test case contains an integer **N** denoting the length of the array **A**.
- The second line contains **N** space-separated integers $\mathbf{A_1}, \mathbf{A_2} \ldots \mathbf{A_N}$.

Output

- For each test case, print two lines.
- The first line should contain the maximum possible Hamming distance that array **B** can have from array **A**.
- The second line should contain **N** space-separated integers denoting the array **B**; the **i**-th integer should denote the value of $\mathbf{B_i}$. Note that **B** should be an array obtained after shuffling the elements of **A**.

Constraints

- $1 \leq \mathbf{T} \leq 10$
- $1 \leq \mathbf{N} \leq 10^5$
- $1 \leq \mathbf{A_i} \leq 10^5$
- The frequency of each integer in the array **A** will be at most 2.

Subtasks

Subtask #1 (30 points): all elements in the array **A** are unique

Subtask #2 (30 points): $5 \leq \mathbf{N} \leq 10^5$

Subtask #3 (40 points): original constraints

Sample Input 1: 3 2 1 2 3 1 2 1 4 2 6 5 2 ### Sample Output 1: 2 2 1 2 2 1 1 4 6 2 2 5 ### Explanation:

Problem Name: Chef and Strange Matrix

Problem Statement:

Spring is interesting season of year. Chef is thinking about different things, but last time he thinks about interesting game - "Strange Matrix".

Chef has a matrix that consists of **n** rows, each contains **m** elements. Initially, the element $\mathbf{a_{ij}}$ of matrix equals **j**. **(1 ≤ i ≤ n, 1 ≤ j ≤ m)**.

Then **p** times some element $\mathbf{a_{ij}}$ is increased by **1**.

Then Chef needs to calculate the following:

- For each row he tries to move from the last element (with number **m**) to the first one (with the number **1**).
- While staying in $\mathbf{a_{ij}}$ Chef can only move to $\mathbf{a_{ij-1}}$ only if $\mathbf{a_{ij-1} \le a_{ij}}$.
- The cost of such a movement is $\mathbf{a_{ij}} - \mathbf{a_{ij-1}}$.
- Otherwise Chef can't move and lose (in this row).
- If Chef can move from the last element of the row to the first one, then the answer is the total cost of all the movements.
- If Chef can't move from the last element of the row to the first one, then the answer is **-1**.

Help Chef to find answers for all the rows after **P** commands of increasing.

Input

- The first line contains three integers **n**, **m** and **p** denoting the number of rows, the number of elements a single row and the number of increasing commands.
- Each of next **p** lines contains two integers **i** and **j** denoting that the element $\mathbf{a_{ij}}$ is increased by one.

Output

- For each row in a single line print the answer after the **P** increasing commands.

Constraints

- **1 ≤ n, m, p ≤ 10 ^ 5**
- **1 ≤ i ≤ n**
- **1 ≤ j ≤ m**

Sample Input 1:

4 4 6 2 2 3 2 3 2 4 3 4 4 4 3

Sample Output 1:

3 3 -1 4

Explanation:

Here is the whole matrix after **P** commands:

1 2 3 4 1 3 3 4 1 4 3 4 1 2 5 5

Explanations to the answer:

- The first line is without changes: 4-3=1, 3-2=1, 2-1=1. answer = 3.
- The second line: 4-3=1, 3-3=0, 3-1=2. The answer is 3.
- The third line: 4-3=1, 3-4=-1, Chef can't move to the first number here. Therefore, the answer is -1.
- The fourth line: 5-5=0, 5-2=3, 2-1=1. The answer is 4.

Problem Name: Chef and String to Integer

Problem Statement:

Chef has found a strange string that may contain spaces, signs, numbers, and letters. He wants to **convert this string into an integer** by following a few simple rules.

You need to help Chef by writing a program that performs this conversion carefully.

Conversion Rules

- **Ignore spaces**:
  - Skip all leading spaces before any character.
- **Check the sign**:
  - If the next character is `'-'`, the number is negative.
  - If the next character is `'+'`, the number is positive.
  - If no sign is given, assume it is positive.
- **Read digits only**:
  - Read all consecutive digits and stop when a non-digit character is found.
  - If no digits are found, the result is **0**.
- **Clamp within range**:
  - If the number is smaller than **-2147483648**, return **-2147483648**.
  - **If the number is larger than 2147483647, return 2147483647.**

Function Declaration

Function Name

myAtoi – This function converts a given string into a 32-bit signed integer by following specific conversion rules similar to the standard `atoi` function.

Parameters

- *s* : A string that may contain leading spaces, optional sign characters (+ or -),digits, and other non-digit characters.

Return Value

- Returns an integer obtained after converting the string according to the given rules.
- If no valid number can be formed, the function returns `0`.
- If the converted number is smaller than `-2147483648`, it returns `-2147483648`.

If the converted number is larger than 2147483647, it returns 2147483647.

Input Format

- The first line contains an integer T — the number of test cases.
- **Each of the next T lines contains one string S.**

Output Format

- **For each test case, print the converted integer on a new line.**

Constraints

- $1 \le T \le 100$
- $0 \le |S| \le 200$
- $\texttttS$ may contain letters, digits, spaces, '+', '-', and '.'

Sample Input 1:

5 98hello -00456 +45abc23 word123 -91283472332

Sample Output 1:

98 -456 45 0 -2147483648

Explanation:

Test case 1:\" 98hello" -> ignore spaces -> read digits 98 -> result = **98**

Test case 2:\"-00456" -> negative sign -> read digits 456 -> result = **-456**

Test case 3:\"+45abc23" -> sign + -> read digits 45 -> result = **45**

Test case 4:\"word123" -> starts with non-digit ? no number found -> result = **0**

Test case 5:\"-91283472332" -> number is too small -> clamp to **-2147483648**

Problem Name: Maximum Subarray

Problem Statement:

Given two arrays A and B of sizes N and M respectively. You can apply the following operation until the array B is non-empty:

- Choose either the **first** or the **last** element of array B.
- Insert the chosen element to either the **front** or the **back** of array A.
- Delete the chosen element from array B.

For example, let $A = [9, 7]$ and $B = [1, 3, 2]$. In one operation, we can choose either $X = 1$ or $X = 2$ (first or last element of array B). We can insert X in array A and make it either $A = [X, 9, 7]$ or $A = [9, 7, X]$. The chosen X is deleted from array B. Thus, it will become either $B = [3, 2]$ (when chosen X is 1) or $B = [1, 3]$ (when chosen X is 2).

Find the **maximum sum** of any subarray of the array A that you can achieve after performing exactly M operations.

Note: A subarray of an array is formed by deleting some (possibly zero) elements from the beginning of the array and some (possible zero) elements from the end of the array. A subarray can be empty as well.

Input Format

- The first line of input will contain a single integer T, denoting the number of test cases.
- Each test case consists of 4 lines of input.
    - The first line of each test contains a single integer N, the size of array A.
    - The next line contains N space-separated integers, denoting elements of array A.
    - The third line of each test contains a single integer M, the size of array B.
    - The next line contains M space-separated integers, denoting elements of array B.

Output Format

For each test case, output on a new line the **maximum sum** of any subarray of the array A that you can achieve after performing exactly M operations.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 10^5$
- $1 \leq M \leq 10^5$
- $-10^8 \leq A_i, B_i \leq 10^8$

Sample Input 1:

3 5 3 26 -79 72 23 2 66 44 1 81 1 -97 5 10 -5 14 -20 4 3 -10 5 -2

Sample Output 1:

205 81 24

Explanation:

Test case *1*:

- Operation *1*: Add the first element of array *B* to the back of array *A*. Thus, *A = [3, 26, –79, 72, 23, 66]* and *B = [44]*.
- Operation *2*: Add the first element of array *B* to the back of array *A*. Thus, *A = [3, 26, –79, 72, 23, 66, 44]* and *B = []*.

The, maximum sum subarray of array *A* is *[72, 23, 66, 44]* having sum *72+23+66+44=205*.

Test case *2*:

- Operation *1*: Add the first element of array *B* to the front of array *A*. Thus, *A = [–97, 81]* and *B = []*.

The, maximum sum subarray of array *A* is *[81]* having sum *81*.

Test case *3*:

- Operation *1*: Add the last element of array *B* to the back of array *A*. Thus, *A = [10, –5, 14, –20, 4, –2]* and *B = [–10, 5]*.
- Operation *2*: Add the last element of array *B* to the front of array *A*. Thus, *A = [5, 10, –5, 14, –20, 4, –2]* and *B = [–10]*.
- Operation *3*: Add the first element of array *B* to the front of array *A*. Thus, *A = [–10, 5, 10, –5, 14, –20, 4, –2]* and *B = []*.

The, maximum sum subarray of array *A* is *[5, 10, –5, 14]* having sum *5+10–5+14 = 24*.